UNIVERSITÄT
BERN

Ipopt revisited: Strategies to solve edge cases

Horacio Lisdero Scaffino

May 21, 2026

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

UNIVERSITÄT BERN

Agenda

A tour of Ipopt
- History
- Techniques applied

An attempt of a simple IPM algorithm
- Challenges encountered
- Results

A tour of ripopt

Experiments
- What can we learn

Conclusion

Ipopt: Interior Point OPTimizer

Popular open-source non-linear optimization package

Bindings available for:

- AMPL (algebraic modeling language)
- Fortran
- GAMS (general algebraic modeling system)
- Julia
- Matlab / Scilab
- .NET languages: C#, F# and Visual Basic.NET
- Python (several wrappers available)
- R
- Rust

Ipopt: The origins

- PhD thesis by Andreas Wächter in 2002 at Carnegie Mellon [1]
- Since 2002 part of the COIN-OP foundation [2]
- Original implementation in Fortran 77 (1.x and 2.x), now deprecated
- Open source since the start
- Benchmarked against KNITRO (1999) and LOQO (1999) [3, 4]

Ipopt: Subsequent paper

- Paper by Wächter and Biegler in 2006 [5]
- More focused on the central algorithm of Ipopt
- Features described more in detail
  - Feasibility restoration phase,
  - Inertia correction,
  - Heuristics...
- More benchmarks: CUTEr test set [6]

Ipopt: C++ version

- IBM Research decided to invest in an open source re-write of Ipopt in C++
- With the help of Carl Laird, the code was re-implemented from scratch [7]
- 3.x published in Aug 2005
- It is still actively developed on GitHub

Summary

- Primal-dual interior-point method with filter line-search globalization
- Sparse symmetric KKT solves with inertia monitoring/correction
- Inertia correction and regularization to obtain the proper search direction
- Feasibility restoration phase for difficult or infeasible iterates
- Second-order correction steps to reduce Maratos-type effects
- Practical heuristics for robustness and faster performance

UNIVERSITÄT BERN

Interior Point Methods

UNIVERSITÄT BERN

Agenda

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

I wanted to contribute something back to the community,

i.e, **not** throw away the Python code 1 week after the seminar

The original plan

1. Select the most popular optimization package
2. Implement a simple interior-point method
3. Compare it with Ipopt
4. Profit!

argmin

argmin v0.11.0

Numerical optimization in pure Rust

Homepage Documentation Repository

All-Time: 3,049,452

Recent: 531,942

Updated: 7 months ago

Website [8]

Pure Rust 🦀

We aim at providing a wide range of solvers implemented in pure Rust.

Consistent interface 🔌

Easily plug your optimization problem in different solvers thanks to a consistent interface.

Type agnostic 🔧

Represent your variables, gradients, Jacobians and Hessians using your own types.

Various math backends 🧮

Use Vecs, ndarray, or nalgebra for behind-the-scenes math. Or implement your own backend.

Observers 👀

Observe the progress of your optimization by logging to screen or file or implement your own observer.

Checkpointing 🚩

Mitigate the negative effects of crashes in unstable computing environments by saving regular checkpoints.

Extensible 🪡

Most features can be exchanged for user supplied implementations thanks to Rusts powerful generics and traits.

Development framework 🏗️

Use argmins tools and ecosystem to implement your own solver – with all the features mentioned above.

discarded several libraries: dealt only with LP (`good_lp`), used novel algorithms (`clarabel`), or were too small (`interiors`)

argmin: Code example

```rust
impl<O, P, G, H, F> Solver<O, IterState<P, G, (), H, (), F>> for Newton<F>
where
    O: Gradient<Param = P, Gradient = G> + Hessian<Param = P, Hessian = H>,
    P: Clone + ArgminScaledSub<P, F, P>,
    H: ArgminInv<H> + ArgminDot<G, P>,
    F: ArgminFloat,
{
    fn next_iter(
        &mut self,
        problem: &mut Problem<O>,
        mut state: IterState<P, G, (), H, (), F>,
    ) -> Result<(IterState<P, G, (), H, (), F>, Option<KV>), Error> {
        let param = state.take_param().ok_or_else(argmin_error_closure!(
            NotInitialized,
            concat!(
                "`Newton` requires an initial parameter vector. ",
                "Please provide an initial guess via `Executor`s `configure` method."
            )
        ))?;
        let grad = problem.gradient(&param)?;
        let hessian = problem.hessian(&param)?;
        let new_param = param.scaled_sub(&self.gamma, &hessian.inv()?.dot(&grad));
        Ok((state.param(new_param), None))
    }
}
```

UNIVERSITÄT BERN

Math backends

The math backend is implemented in a separate crate `argmin-math`

It is trait-based: interfaces define the methods required from the linear algebra backend

- `nalgebra` [9]
- `faer` [10]

Add a solver

Add a trait:

```rust
pub trait ArgminLuSolve<T> {
    /// Solve linear system using LU decomposition.
    fn lu_solve(&self, rhs: &T) -> Result<T, Error>;
}
```

And the corresponding implementation:

```rust
impl<N, D, C, S> ArgminLuSolve<OMatrix<N, D, C>> for SquareMatrix<N, D, S>
where
    N: ComplexField,
    D: Dim + DimMin<D, Output = D>,
    C: Dim,
    S: Storage<N, D, D>,
    DefaultAllocator: Allocator<N, D, D> + Allocator<N, D, C> + Allocator<N, D>,
{
    #[inline]
    fn lu_solve(&self, rhs: &OMatrix<N, D, C>) -> Result<OMatrix<N, D, C>, Error> {
        let lu = LU::new(self.clone_owned());
        lu.solve(rhs).ok_or_else(|| LuSolveError {}.into())
    }
}
```

UNIVERSITÄT BERN

No constraints?

```rust
/// Defines equality constraints for a problem.
///
/// It follows the form h(x) = (0, ... , 0), where h is a vector field.
pub trait EqualityConstraint {
    /// Type of the parameter vector
    type Param;
    /// Type of the return value of the equality constraints
    type Output;

    /// Compute all equality constraints
    fn equality_constraint(&self, param: &Self::Param) -> Result<Self::Output, Error>;
}
```

And their derivatives...

```
/// Defines the Jacobian of the vector field representing the equality constraints for a
↪ problem.
pub trait EqualityConstraintJacobian {
    /// Type of the parameter vector
    type Param;
    /// Type of the Jacobian
    type Jacobian;

    /// Compute the Jacobian of the equality constraints
    fn equality_constraint_jacobian(&self, param: &Self::Param) -> Result<Self::Jacobian,
    ↪ Error>;

    bulk!(equality_constraint_jacobian, Self::Param, Self::Jacobian);
}
```

And the Hessian of the Lagrangian...

```
pub trait LagrangianHessian {
    /// Type of the parameter vector
    type Param;
    /// Type of the multipliers associated with equality constraints
    type MultipliersEq;
    /// Type of the multipliers associated with inequality constraints
    type MultipliersIneq;
    /// Type of the Hessian
    type Hessian;

    /// Compute the Hessian of the Lagrangian
    fn lagrangian_hessian(
        &self,
        param: &Self::Param,
        multipliers_eq: &Self::MultipliersEq,
        multipliers_ineq: &Self::MultipliersIneq,
    ) -> Result<Self::Hessian, Error>;

    bulk!(
        lagrangian_hessian,
        Self::Param,
        Self::MultipliersEq,
        Self::MultipliersIneq,
        Self::Hessian
    );
}
```

And iteration state for NLP...

- param: Option<P>
- slacks: Option<S>
- best_param: Option<P>
- best_slacks: Option<S>
- cost: F
- best_cost: F
- target_cost: F
- grad: Option<G>
- lagrangian_hessian: Option<LH>
- equality_constraints: Option<EqC>
- inequality_constraints: Option<IneqC>
- equality_constraint_jacobian: Option<EqCJ>
- inequality_constraint_jacobian: Option<IneqCJ>
- lambda_eq: Option<LambdaEq>
- lambda_ineq: Option<LambdaIneq>
- alpha_primal: Option<F>
- alpha_dual: Option<F>
- inf_pr: Option<F>
- inf_du: Option<F>
- compl_inf: Option<F>
- iter: u64
- last_best_iter: u64
- max_iters: u64
- counts: HashMap<String, u64>
- counting_enabled: bool
- time: Option<Duration>
- termination_status: TerminationStatus

Finally! IPM

`nalgebra_backend.rs`: All the linear-algebra-specific logic for nalgebra, the only supported backend

`linesearch.rs`: The custom backtracking linear search. The existing implementation could not be reused

`iteration.rs`: A single iteration of the interior-point method

`interiorpointmethod.rs`: The solver. The high-level management of the IPM algorithm.

UNIVERSITÄT BERN

Agenda

ripopt: A Rust re-write from February 2026

Coded in 60/70 hs during the course of 6 weeks heavily relying on Claude Code [11].

According to the benchmarks CUTEst and xxxx, it already beats Ipopt.

Interesting insights into licensing issues and thought process documented in the repo.

Agenda

UNIVERSITÄT BERN

Agenda

Conclusion

Ipopt's performance can still be improved.

Different methods complement each other: By combining them, edge cases can be solved more efficiently.

Implementation of a state-of-the-art optimization package may be in the reach of modern AI agentic coding tools.

Questions?

Links

Presentation: https://github.com/hlisdero/JMCS_Seminar_Applied_Optimization

Modified argmin: https://github.com/hlisdero/argmin

ripopt: https://github.com/jkitchin/ripopt

ipopt-rs: https://github.com/elrnv/ipopt-rs

Bibliography

A. Wachter, *An interior point algorithm for large-scale nonlinear optimization with applications in process engineering*. Carnegie Mellon University, 2002.

C.-O. Foundation, "Coin-op ipopt." https://github.com/coin-or/Ipopt. Accessed: 2026-04-15.

R. H. Byrd, M. E. Hribar, and J. Nocedal, "An interior point algorithm for large-scale nonlinear programming," *SIAM Journal on Optimization*, vol. 9, no. 4, pp. 877–900, 1999.

R. J. Vanderbei, "Loqo: An interior point code for quadratic programming," *Optimization methods and software*, vol. 11, no. 1-4, pp. 451–484, 1999.

A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.

N. I. Gould, D. Orban, and P. L. Toint, "Cuter and sifdec: A constrained and unconstrained testing environment, revisited," *ACM Transactions on Mathematical Software (TOMS)*, vol. 29, no. 4, pp. 373–394, 2003.

COIN-OR Foundation, *Ipopt: Interior Point OPTimizer*, 2026.

S. Kroboth, "argmin: Mathematical optimization in pure rust." https://argmin-rs.org/. Accessed: 2026-03-28.

S. Crozet, "nalgebra: Linear algebra library for rust." https://nalgebra.rs/. Accessed: 2026-03-28.

S. Quiñones, "faer: linear algebra library for rust." https://faer.veganb.tw/.